Load Balanced Architecture with Advanced Request Routing

Simple step-by-step guide (EC2 + S3 + IAM + ALB + Route 53)

Prepared by: **Sukhada Wagale** | Date: **16 October 2026**

1. What you will build

Two small websites, **Red** and **Blue**, run on two EC2 instances in different Availability Zones behind one Application Load Balancer (ALB). The ALB sends each request to the right instance using:

- **Path-based routing:** `/red` goes to the Red instance and `/blue` goes to the Blue instance.
- **Host-based routing:** `red.yourdomain.com` goes to Red and `blue.yourdomain.com` goes to Blue.

Exercise	What you do
1	Create S3 bucket, security group, IAM role; launch the Red and Blue EC2 instances
2	Create target groups and the ALB; add path-based rules
3	Replace with host-based rules; create Route 53 records
4	Clean up all resources

Before you start

- AWS account, Region **us-east-1 (N. Virginia)** for the whole lab.
- The lab files from the course: `advanced-request-routing-code.zip`. Unzip it. It contains the website files, `user-data-red`, `user-data-blue` and `bucket-permissions.json`.
- For Exercise 3 only: a domain name registered in Route 53 (see the note in Exercise 3 if you do not have one).

2. Exercise 1 - Create the Red and Blue EC2 instances

Task 1: Create the S3 bucket and upload the code

1. Open the **S3** console > **Create bucket**.
2. Bucket name: `arr-bucket-123456` (change the numbers so the name is globally unique). **Write the name down.**
3. Keep the defaults and click **Create bucket**.
4. Open the bucket > **Upload** > add **all files from the zip except** the two user-data files and `bucket-permissions.json` > **Upload**.

Task 2: Create the security group

1. Open **EC2** > **Security Groups** > **Create security group**.
2. Name: `WebsiteSG`. Description: `WebsiteSG`. VPC: the default VPC.
3. Inbound rule: Type **HTTP**, Port 80, Source **Anywhere-IPv4 (0.0.0.0/0)**. Click **Create security group**.

Task 3: Create the IAM policy and role

1. Open **IAM** > **Policies** > **Create policy** > **JSON** tab.
2. Paste the contents of `bucket-permissions.json` and replace `YOUR-BUCKET-ARN` with your bucket's ARN (S3 > your bucket > Properties > Bucket ARN).
3. Click **Next**, name it `S3-ARR-Policy` and click **Create policy**.
4. **Roles** > **Create role** > Trusted entity **AWS service**, use case **EC2** > **Next**.
5. Attach `S3-ARR-Policy` > **Next**, name the role `S3-ARR-Role` > **Create role**.

> **Why:** the EC2 instances copy the website files from S3 when they start. The role gives them permission to read your bucket without storing any access keys on the server.

Task 4: Launch the Red instance

1. **EC2** > **Launch instances**. Name: **Red**.
2. AMI: **Amazon Linux 2023**. Instance type: **t2.micro** (use t3.micro if t2.micro is not offered).
3. Key pair: **Proceed without a key pair**.
4. Network settings > **Edit**: Subnet **us-east-1a**, Auto-assign public IP **Enable**, select existing security group **WebsiteSG**.
5. **Advanced details** > IAM instance profile: **S3-ARR-Role**.
6. In **User data** paste the contents of `user-data-red` and change every `YOUR-BUCKET-NAME` to your bucket name.
7. Click **Launch instance**.

What the user data does (for reference)

```
#!/bin/bash
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
mkdir /var/www/html/red
cd /var/www/html/red
aws s3 cp s3://YOUR-BUCKET-NAME/hw-red.css ./
aws s3 cp s3://YOUR-BUCKET-NAME/hw-red-py.css ./
aws s3 cp s3://YOUR-BUCKET-NAME/python.png ./
aws s3 cp s3://YOUR-BUCKET-NAME/apache.svg ./
aws s3 cp s3://YOUR-BUCKET-NAME/red-index.html ./index.html
cd /var/www/html
(same files copied again; red-root-index.html becomes index.html)
systemctl restart httpd
```

Task 5: Launch the Blue instance

1. Repeat Task 4 with Name **Blue**, subnet **us-east-1b**, the same security group and IAM role.
2. Use the contents of `user-data-blue` (with your bucket name) as user data.
3. Wait 2-3 minutes. Copy each instance's public IPv4 address and test in the browser (http, not https):

```
http://<Red public IP>/red
http://<Blue public IP>/blue
```

If the page is missing: the files were not copied from S3. Check that the IAM role is attached to the instance, the bucket name in user data is exact, and the files were uploaded to the bucket.

3. Exercise 2 - Enable path-based routing

Task 1: Create the target groups

1. **EC2** > **Target Groups** > **Create target group**. Target type: **Instances**.
2. Name: **Red**. Protocol HTTP, port 80, default VPC. Under Health checks set path to `/red/index.html`. Click **Next**.
3. Select the **Red** instance > **Include as pending below** > **Create target group**.
4. Repeat for **Blue**: health check path `/blue/index.html` and register the **Blue** instance.

Task 2: Create the Application Load Balancer

1. **EC2** > **Load Balancers** > **Create load balancer** > **Application Load Balancer**.
2. Name: `LabLoadBalancer`. Scheme: **Internet-facing**.
3. Network mapping: default VPC and tick **both us-east-1a and us-east-1b**.
4. Security group: **WebsiteSG** (remove the default one).
5. Listener **HTTP : 80**, default action **Forward to Blue**.
6. Click **Create load balancer** and wait until the state is **Active**.

Task 3: Add the path rules

1. Open the ALB > **Listeners and rules** > select **HTTP:80** > **Manage rules** > **Add rule**.
2. Rule 1: Condition **Path** = `/red*`, Action **Forward to Red**, Priority **1**.

3. Rule 2: Condition **Path** = `/blue*`, Action **Forward to Blue**, Priority **2**.
4. Wait 1-2 minutes until both target groups show **healthy** targets.
5. Copy the ALB **DNS name** and test `http://<ALB-DNS>/red` and `http://<ALB-DNS>/blue`.

Priority	Condition	Action
1	Path is /red*	Forward to Red
2	Path is /blue*	Forward to Blue
Default	No other rule matches	Forward to Blue

4. Exercise 3 - Enable host-based routing

Task 1: Replace the path rules with host rules

1. Open the ALB listener > **Manage rules** > **Edit rules** and **delete both path rules**.
2. Add rule: Condition **Host header** = `red.yourdomain.com`, Action **Forward to Red**, Priority **1**.
3. Add rule: Condition **Host header** = `blue.yourdomain.com`, Action **Forward to Blue**, Priority **2**.

Task 2: Create the Route 53 records

1. Open **Route 53** > **Hosted zones** > your domain > **Create record**.
2. Record name: **red**. Type **A**. Turn **Alias** on. Route traffic to **Alias to Application and Classic Load Balancer**, Region **US East (N. Virginia)**, choose **LabLoadBalancer**. Create.
3. Repeat for record name **blue**.
4. Create one more A alias record with an **empty name** (apex domain) pointing to the same ALB.
5. Wait a minute or two, then browse to `http://red.yourdomain.com` and `http://blue.yourdomain.com`. You should see the Red and Blue pages.

No domain? You can still test host-based routing from a terminal by sending the Host header yourself: `curl -H "Host: red.yourdomain.com" http://<ALB-DNS>/red/` (use any name you typed in the rule). The response should come from the matching target group.

5. Troubleshooting

Problem	What to check
Target shows unhealthy	Health check path is exactly /red/index.html or /blue/index.html; the user data finished; WebsiteSG allows HTTP 80.
503 from the ALB	No healthy targets in the target group the rule points to. Register the instance and wait for health checks.
504 / timeout	ALB or instance security group missing HTTP 80; ALB not in public subnets.
Web page missing on instance	IAM role not attached, wrong bucket name in user data, or files not uploaded to S3.
Host rule does not match	Typo in the host header value; DNS record not created or still propagating; you used https instead of http.
Always lands on Blue	That is the default action. Rules did not match; check the path/host values and priorities.

6. Exercise 4 - Clean up

1. Delete the **Application Load Balancer** (chargeable).
2. Delete the **Target groups** Red and Blue.
3. **Terminate** the Red and Blue EC2 instances (chargeable).
4. Delete the security group **WebsiteSG** after the instances show Terminated.
5. S3: **Empty** the bucket first, then **Delete** it (chargeable).
6. Route 53: delete the red, blue and apex alias records (leave the NS and SOA records).
7. Optional tidy-up: delete the IAM role **S3-ARR-Role** and policy **S3-ARR-Policy**.

7. What this project teaches

- Using an IAM role so EC2 can read from S3 without access keys.
- Bootstrapping servers automatically with User Data.
- Target groups, health checks and listener rules on an Application Load Balancer.
- Path-based vs host-based routing, and pointing subdomains to an ALB with Route 53 alias records.